CSRF cont'd, Certificates, Password Hashing

CS 161 Spring 2026 - Lecture 18

Review: Cookies and Session Tokens

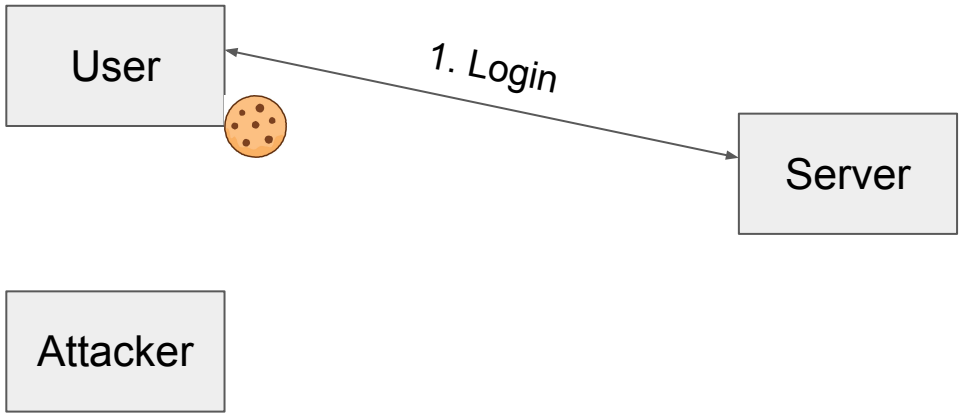
- Session token cookies are used to associate a request with a user
- The browser automatically attaches relevant cookies in every request

Steps of a CSRF Attack



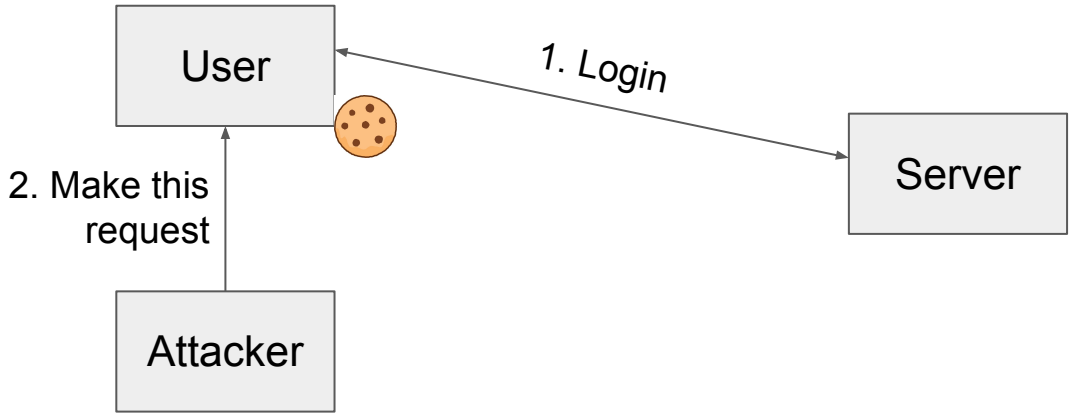
Steps of a CSRF Attack

1. User authenticates to the server
    - User receives a cookie with a valid session token



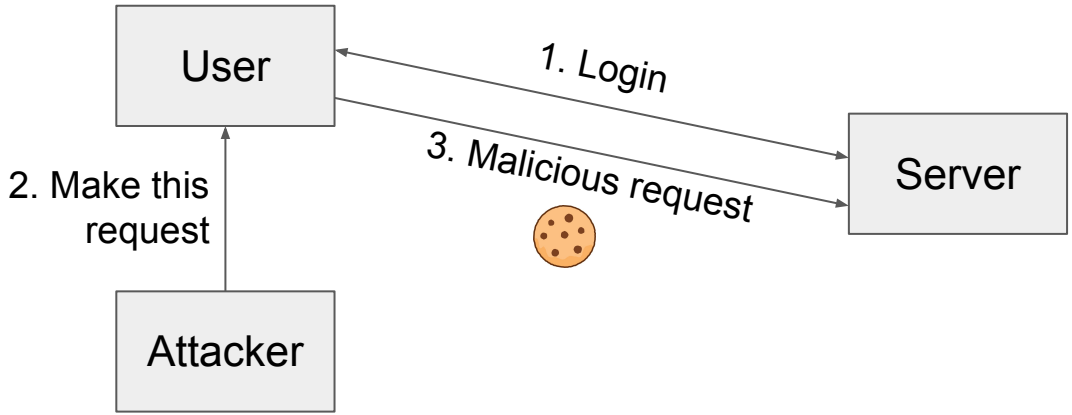
Steps of a CSRF Attack

1. User authenticates to the server
    - User receives a cookie with a valid session token
2. Attacker tricks the victim into making a malicious request to the server



Steps of a CSRF Attack

1. User authenticates to the server
   - User receives a cookie with a valid session token
2. Attacker tricks the victim into making a malicious request to the server
3. The server accepts the malicious request from the victim
   - Recall: The cookie is automatically attached in the request



Steps of a CSRF Attack

1. User authenticates to the server
    - User receives a cookie with a valid session token
2. Attacker tricks the victim into making a malicious request to the server
3. The server accepts the malicious request from the victim
    - Recall: The cookie is automatically attached in the request

Executing a CSRF Attack

- How might we trick the victim into making a GET request?

Executing a CSRF Attack

- How might we trick the victim into making a GET request?
- Strategy #1: Trick the victim into clicking a link
  - For example, send the victim an email with an enticing-looking link
  - The link can directly make a GET request:

    `https://www.bank.com/transfer?amount=100&to=Mallory`
  - The link can open an attacker's website, which contains some JavaScript that makes the actual malicious request
- Strategy #2: Put some HTML on a website the victim will visit
  - Example: The victim will visit a forum. Make a post with some HTML on the forum
  - HTML to automatically make a GET request to a URL:

    `<img src="https://www.bank.com/transfer?amount=100&to=Mallory">`
    - This HTML will probably return an error or a blank 1 pixel by 1 pixel image, but the GET request will still be sent...with the relevant cookies!

Executing a CSRF Attack

- How might we trick the victim into making a POST request?
  - Example POST request: Submitting a form
- Strategy #1: Trick the victim into clicking a link
  - Note: Clicking a link in your browser makes a GET request, not a POST request, so the link cannot directly make the malicious POST request
  - The link can open an attacker's website, which contains some JavaScript that makes the actual malicious POST request
- Strategy #2: Put some JavaScript on a website the victim will visit
  - Example: Pay for an advertisement on the website, and put JavaScript in the ad
  - Recall: JavaScript can make a POST request

CSRF Defenses



CSRF Defenses

- CSRF defenses are implemented by the server (not the browser)
  - Unlike the cookie policy and same origin policy which are enforced by the browser
- Defense: CSRF tokens
- Defense: SameSite cookie attribute

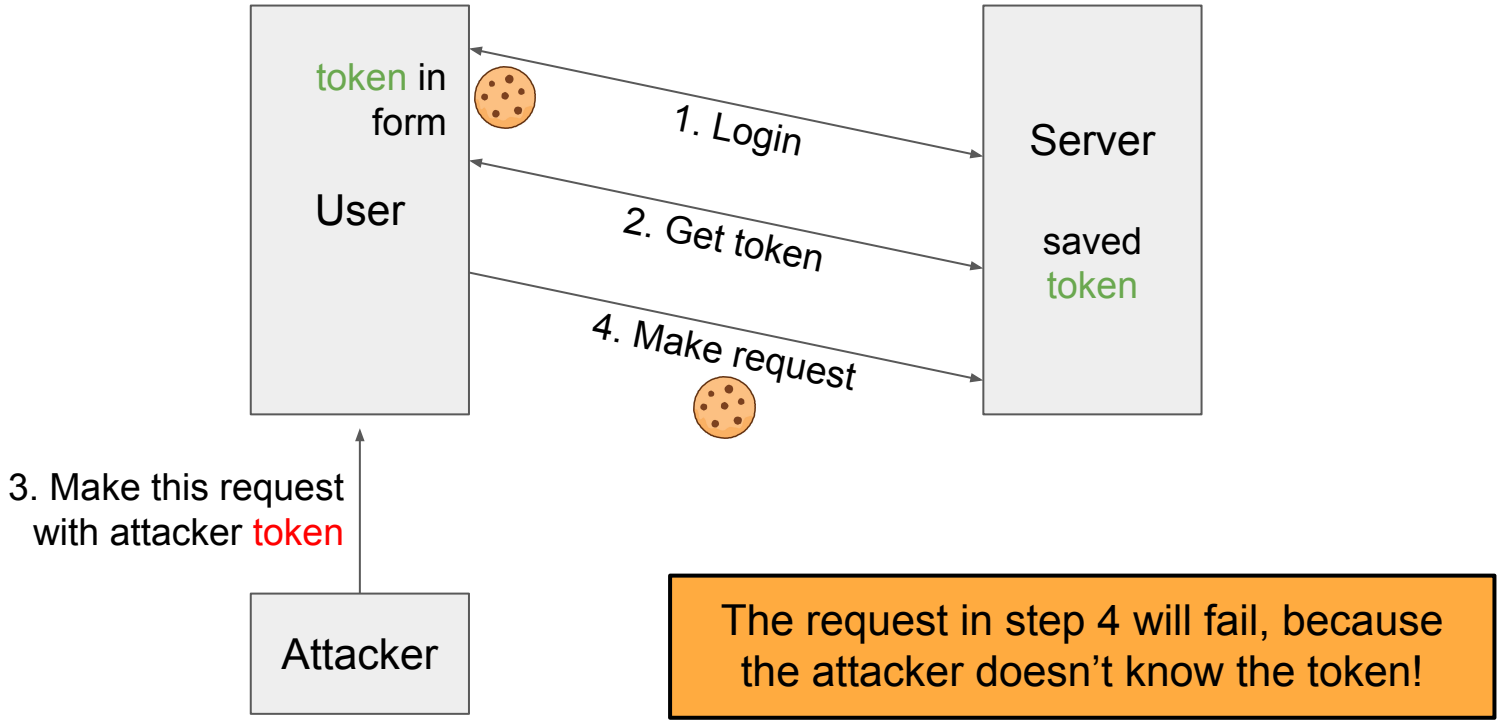
CSRF Tokens

- Idea: Add a secret value in the request that the attacker doesn't know
  - The server only accepts requests if it has a valid secret
  - Now, the attacker can't create a malicious request without knowing the secret
- **CSRF token**: A secret value provided by the server to the user. The server only accepts the request is this value is attached.
  - CSRF tokens cannot be sent to the server in a cookie!
  - The token must be sent somewhere else (e.g., a header, GET parameter, or POST content)

CSRF Tokens: Usage

- Example: HTML forms
  - Forms are vulnerable to CSRF
    - If the victim visits the attacker's page, the attacker's JavaScript can make a POST request with a filled-out form
- CSRF tokens are a defense against this attack
  - Every time the user requests a form from the legitimate website, the server attaches a CSRF token as a *hidden form field* (in the HTML, but not visible to the user)
  - When the user submits the form, the request contains all form fields – including the CSRF token
  - The attacker's JavaScript won't be able to create a valid form, because they don't know the CSRF token!
  - The attacker can try to fetch their own CSRF token, but it will only be valid for the attacker, not the victim

CSRF Tokens: Usage



The request in step 4 will fail, because the attacker doesn't know the token!

Referer Header

- Idea: In a CSRF attack, the victim usually makes the malicious request from a different website
- Referer header: A header in an HTTP request that indicates which webpage made the request
  - “Referer” is a 30-year typo in the HTTP standard (supposed to be “Referrer”)!
  - Example: If you type your username and password into the Facebook homepage, the Referer header for that request is `https://www.facebook.com`
  - Example: If an `img` HTML tag on a forum forces your browser to make a request, the Referer header for that request is the forum’s URL
  - Example: If JavaScript on an attacker’s website forces your browser to make a request, the Referer header for that request is the attacker’s URL

Referer Header

- Checking the Referer header
  - Allow **same-site requests**: The Referer header matches an expected URL
    - Example: For a login request, expect it to come from `https://bank.com/login`
  - Disallow **cross-site requests**: The Referer header does not match an expected URL
- If the server sees a cross-site request, reject it

Referer Header: Issues

- The Referer header may leak private information
  - Example: If you made the request on a top-secret website, the Referer header might show you visited `http://intranet.corp.apple.com/projects/iphone/competitors.html`
  - Example: If you make a request to an advertiser, the Referer header gives the advertiser information about how you saw the ad
- The Referer header might be removed before the request reaches the server
  - Example: Your company firewall removes the header before sending the request
  - Example: The browser removes the header because of your privacy settings
- The Referer header is optional. What if the request leaves the header blank?
  - Allow requests without a header?
    - Less secure: CSRF attacks might be possible
  - Deny requests without a header?
    - Less usable: Legitimate requests might be denied
  - Need to consider fail-safe defaults: No clear answer

SameSite Cookie Attribute

- Idea: Implement a flag on a cookie that makes it unexploitable by CSRF attacks
  - This flag must specify that **cross-site** requests will not contain the cookie
- **SameSite=Strict flag**: An attribute of a cookie that specifies it should be sent only when the domain of the cookie **exactly** matches the domain of the origin of the webpage that triggered the request
  - Example: If **`https://evil.com/`** causes your browser to make a request to **`https://bank.com/transfer?to=mallory`**, cookies for **`bank.com`** will not be sent if SameSite=Strict, because the origin domain (**`evil.com`**) and cookie domain (**`bank.com`**) are different
- Issue: Not yet implemented on some less-common browsers

CSRF Defenses: Summary

- CSRF token: A secret value provided by the server to the user. The user must attach the same value in the request for the server to accept the request.
  - The attacker does not know the token when tricking the user into making a request
- Referer Header: Allow same-site requests, but disallow cross-site requests
  - Header may be blank or removed for privacy reasons
- Same-site cookie attribute: The cookie is sent only when the domain of the cookie exactly matches the domain of the origin
  - Not implemented on all browsers

Today

- Certificates: How do we distribute public keys securely?
- Password Hashing: How do we securely store passwords?
- Case studies

Certificates

Textbook Chapter 13

Review: Public-Key Cryptography

- Public-key cryptography is great! We can communicate securely without a shared secret
  - Public-key encryption: Everybody encrypts with the public key, but only the owner of the private key can decrypt
  - Digital signatures: Only the owner of the private key can sign, but everybody can verify with the public key
- What's the catch?

Problem: Distributing Public Keys

- Public-key cryptography alone is not secure against man-in-the-middle attacks
- Scenario
  - Alice wants to send a message to Bob
  - Alice asks Bob for his public key
  - Bob sends his public key to Alice
  - Alice encrypts her message with Bob’s public key and sends it to Bob
- What can Mallory do?
  - Replace Bob's public key with Mallory’s public key
  - Now Alice has encrypted the message with Mallory’s public key, and Mallory can read it!

Problem: Distributing Public Keys

- Public-key cryptography alone is not secure against man-in-the-middle attacks
- Scenario
  - Alice wants to send a message to Bob
  - Alice asks Bob for his public key
  - Bob sends his public key to Alice
  - Alice encrypts her message with Bob's public key and sends it to Bob
- What can Mallory do?

Problem: Man-in-the-Middle Attack



Alice

Mallory



Bob



Generate PK_B, SK_B

Send PK_B

Send PK_M

Receive PK_M

Send $\{M\}_{PK_M}$

Decrypt $\{M\}_{PK_M}$

Send $\{M\}_{PK_B}$

Decrypt $\{M\}_{PK_B}$

Problem: Distributing Public Keys

- Idea: Sign Bob's public key to prevent tampering
- Problem
  - If Bob signs his public key, we need his public key to verify the signature
  - But Bob's public key is what we were trying to verify in the first place!
  - Circular problem: Alice can never trust any public key she receives
- You cannot gain trust if you trust nothing. You need a root of trust!
  - **Trust anchor**: Someone that we trust completely
  - From our trust anchor, we can begin to trust others

Certificates

- **Certificate**: A signed endorsement of someone's public key
  - A certificate contains at least two things: The **identity** of the person, and their **public key**
- Abbreviated notation
  - Encryption under a public key *PK*: {"Message"}$_{PK}$
  - Signing with a private key *SK*: {"Message"}$_{SK^{-1}}$
    - Recall: A signed message must contain the message along with the signature; you can't send the signature by itself!
- Scenario: Alice wants Bob's public key. Alice trusts EvanBot (PK_E, SK_E)
  - EvanBot is our trust anchor
  - If we trust PK_E, a certificate we would trust is {"Bob's public key is PK_B"}$_{SK_E^{-1}}$

Certificates: how you could have come up with them

- Suppose Alice wants Bob's key. Alice trusts EvanBot (PK_E, SK_E)
- Alice -> EvanBot: "What is Bob's public key?"
- EvanBot -> Alice: {“Bob’s public key is PK_B”}$_{SK_E^{-1}}$

Attempt #1: The Trusted Directory

- Idea: Make a central, trusted directory (TD) from where you can fetch anybody's public key
  - The TD has a public/private keypair PK_{TD}, SK_{TD}
  - The directory publishes PK_{TD} so that everyone knows it (baked into computers, phones, OS, etc.)
  - When you request Bob's public key, the directory sends a certificate for Bob's public key
    - {"Bob's public key is PK_B"}$_{SK_{TD}^{-1}}$
  - If you trust the directory, then now you trust every public key from the directory
- What do we have to trust?

Attempt #1: The Trusted Directory

- Idea: Make a central, trusted directory (TD) from where you can fetch anybody's public key
  - The TD has a public/private keypair PK_{TD}, SK_{TD}
  - The directory publishes PK_{TD} so that everyone knows it (baked into computers, phones, OS, etc.)
  - When you request Bob's public key, the directory sends a certificate for Bob's public key
    - {"Bob's public key is PK_B"}$_{SK_{TD}^{-1}}$
  - If you trust the directory, then now you trust every public key from the directory
- What do we have to trust?
  - We have received TD's key correctly
  - TD won't sign a key without verifying the identity of the owner

Attempt #1: The Trusted Directory

- Let's say that Michael Drake (MD, President of UC) runs the TD
  - We want Nick Weaver's public key: Ask MD
  - We want David Wagner's public key: Ask MD
  - We want Raluca Ada Popa's public key: Ask MD
  - We want Peyrin Kao's public key: Ask MD
  - MD has better things to do (like making sure his private key isn't stolen)!
- Problems?

Attempt #1: The Trusted Directory

- Let's say that Michael Drake (MD, President of UC) runs the TD
  - We want Nick Weaver's public key: Ask MD
  - We want David Wagner's public key: Ask MD
  - We want Raluca Ada Popa's public key: Ask MD
  - We want Peyrin Kao's public key: Ask MD
  - MD has better things to do (like making sure his private key isn't stolen)!
- Problems: Scalability
  - One directory won't have enough compute power to serve the entire world
- Problem: Single point of failure
  - If the directory fails, *cryptography stops working*
  - If the directory is compromised, you can't trust anyone
  - If the directory is compromised, it is difficult to recover

Certificate Authorities

- Addressing scalability: Hierarchical trust
  - The roots of trust may **delegate** trust and signing power to other authorities
    - {“Rich Lyons’ public key is PK_{RL}, and I trust her to sign for UCB”}$_{SK_{MD}^{-1}}$
    - {“Jelani Nelson’s public key is PK_{JN}, and I trust him to sign for the CS department”}$_{SK_{RL}^{-1}}$
    - {“Raluca Ada Popa’s public key is PK_{RAP} (but I don’t trust her to sign for anyone else)”}$_{SK_{JN}^{-1}}$
    - {“Henry CG's public key is PK_{HCG} (but I don’t trust himg to sign for anyone else)”}$_{SK_{JN}^{-1}}$
  - MD is still the root of trust (**root certificate authority**, or **root CA**)
  - RL and JN receive delegated trust (**intermediate CAs**)
  - RAP’s identity can be trusted
- Addressing scalability: Multiple trust anchors
  - There are ~150 root CAs who are implicitly trusted by most devices
  - Public keys are hard-coded into operating systems and devices
  - Each delegation step can restrict the scope of a certificate’s validity
  - Creating the certificates is an *offline* task: The certificate is created once in advance, and then served to users when requested

Revocation

- What happens if a certificate authority messes up and issues a bad certificate?
  - Example: {“Bob’s public key is PK_M”}$_{SK_{CA}^{-1}}$
  - Example: Verisign (a certificate authority) accidentally issued a certificate saying that an average Internet user’s public key belonged to Microsoft
- How could we design revocation?

Revocation: Expiration Dates

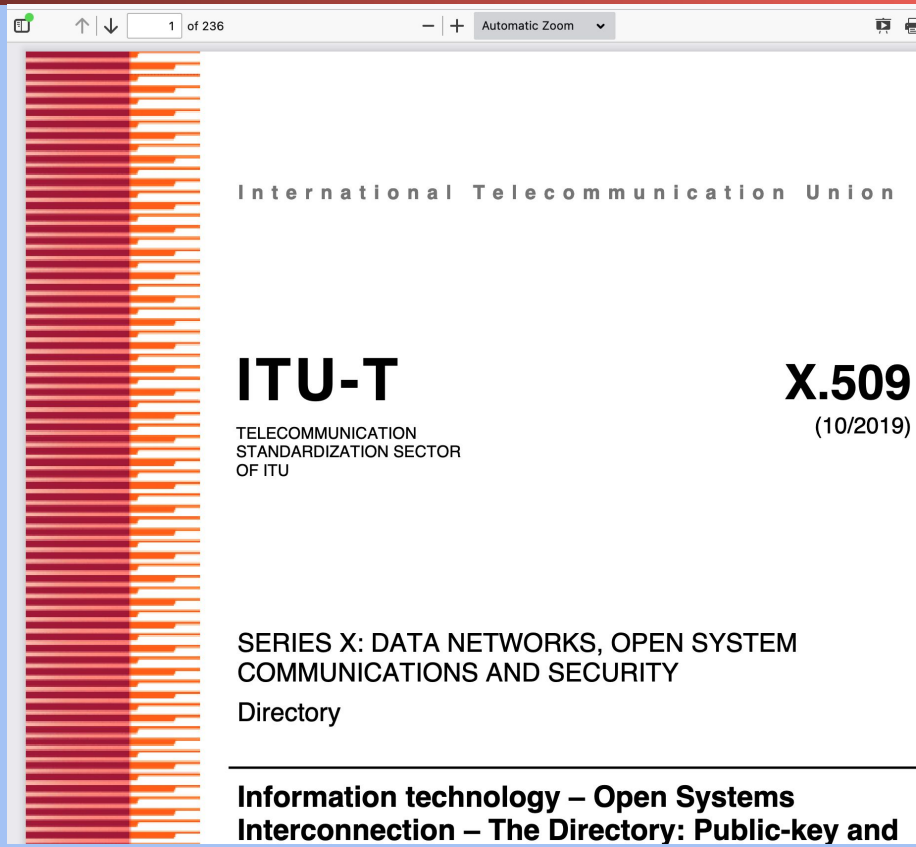
- Approach #1: Each certificate has an expiration date
  - When the certificate expires, request a new certificate from the certificate authority
  - The bad certificate will eventually become invalid once it expires
- Benefits
  - Mitigates damage: Eventually, the bad certificate will become harmless
- Drawbacks
  - Adds management burden: Everybody has to renew their certificates frequently
  - If someone forgets to renew a certificate, their website might stop working
- Tradeoff: How often should certificates be renewed?
  - Frequent renewal: More secure, less usable
  - Infrequent renewal: Less secure, more usable
- LetsEncrypt (a certificate authority) chose very frequent renewal
  - It turns out frequent renewal is more usable:
    It forces automated renewal instead of a once-every 3 year task that gets forgotten!

Revocation: Announcing Revoked Certificates

- Approach #2: Periodically release a list of invalidated certificates
  - Users must periodically download a Certification Revocation List (CRL)
- How do we authenticate the list?
  - The certificate authority signs the list!
    - {“The certificate with serial number 0xdeadbeef is now revoked”}$_{SK_{CA}^{-1}}$
- Drawbacks
  - Lists can get large
    - Mitigated by shorter expiration dates (don’t have to list them once they expire)
  - Until a user downloads a list, they won’t know which certificates are revoked
- What happens if the certificate authority is unavailable?
  - Fail-safe default: Assume all certificates are invalid? Now we can’t trust anybody!
    - Possible attack: Attacker forces the CA to be unavailable (denial of service attack)
  - Use old list: Potentially dangerous if the old list is missing newly revoked certificates

Certificates: Complexity

- Certificate protocols can get very complicated
  - Example: X.509 is incredibly complicated (a 236 page standard!) because it tried to do everything



Summary: Certificates

- Certificates: A signed attestation of identity
- Trusted directory: One server holds all the keys, and everyone has the TD's public key
  - Not scalable: Doesn't work for billions of keys
  - Single point of failure: If the TD is hacked or is down, cryptography is broken
- Certificate authorities: Delegated trust from a pool of multiple root CAs
  - Root CAs can sign certificates for intermediate CAs
  - Revocation: Certificates contain an expiration date
  - Revocation: CAs sign a list of revoked certificates

Trust-on-First-Use

- **Trust-on-first-use**: The first time you communicate, trust the public key that is provided and warn the user if it changes in the future
  - Used in SSH and a couple other protocols
  - Idea: Attacks aren't frequent, so assume that you aren't being attacked the first time
  - Also known as "**Leap of Faith**"

Password Hashing

Textbook Chapter 14

Review: Cryptographic Hashes

- Hash functions accept arbitrarily large inputs
- Hashes “look” random
  - Change a single bit on the input and each output bit has a 50% chance of flipping
  - And until you change the input, you can't predict which output bits are going to change
- Hash functions are *fast*
  - Can operate at 100 MB/s - 3 GB/s
- Security properties
  - One way: For x random, given an output y, it is infeasible to find any input x such that $H(x) = y$.
  - Collision resistant: It is infeasible to find any pair of inputs $x' \neq x$ such that $H(x) = H(x')$.

Storing Passwords

- Password: A secret string a user types in to prove their identity
  - When you create an account with a service: Create a password
  - When you later want to log in to the service: Type in the same password again
- How does the service check that your password is correct?
- Bad idea #1: Store a file listing every user's password
  - Problem: What if an attacker hacks into the service? Now the attacker knows everyone's passwords!
- Bad idea #2: Encrypt every user's password before storing it, and store the decryption key at the server
  - Problem: The attacker could steal the passwords file *and* the key and decrypt everyone's passwords!
- We need a way to verify passwords *without* storing any information that would allow someone to recover the original password

Password Hashing

- For each user, store a *hash* of their password
- Verification process
  - Hash the password submitted by the user
  - Check if it matches the password hash in the file
- What properties do we need in the hash?
  - Deterministic: To verify a password, it has to hash to the same value every time
  - One-way: We don’t want an attacker who hacks the server and downloads all password hashes to be able to reverse hashes into original passwords

Password Hashing: Attacks

- What if two different users decide to use **`password123`** as their password?
  - Hashes are deterministic: They'll have the same password hash
  - An attacker can see which users are using the same password
- Brute-force attacks
  - Most people use insecure, common passwords
  - An attacker can pre-compute hashes for common passwords: *H*(**`"password123"`**), *H*(**`"password1234"`**), *H*(**`"1234567890"`**), etc.
  - **Dictionary attack**: Hash an entire dictionary of common passwords
- **Rainbow tables**: A pre-computed table of brute-forced passwords that enables the attacker to more effectively identify passwords

Password	Hash
abcdefg000	0x3232ab…
password123	0x52b2d4…

Salted Hashes

- Solution #1: Add a unique, random salt for each user
- **Salt**: A random, public value designed to make brute-force attacks harder
  - For each user, store at the server: username, salt, *H*(password || salt)
  - To verify a user's password: look up their salt in the passwords file, compute *H*(password || salt), and check it matches the hash in the file
  - Salts should be long and random
  - Salts are not secret (think of them like nonces or IVs)
- Brute-force attacks are now harder
  - Assume there are *M* possible passwords and *N* users in the database
  - Unsalted database: Hash all possible passwords, then lookup all users' hashes ⇒
  - Salted database: Hash all passwords for each user's salt ⇒

Salted Hashes

- Solution #1: Add a unique, random salt for each user
- **Salt**: A random, public value designed to make brute-force attacks harder
  - For each user, store at the server: username, salt, *H*(password || salt)
  - To verify a user's password: look up their salt in the passwords file, compute *H*(password || salt), and check it matches the hash in the file
  - Salts should be long and random
  - Salts are not secret (think of them like nonces or IVs)
- Brute-force attacks are now harder
  - Assume there are *M* possible passwords and *N* users in the database
  - Unsalted database: Hash all possible passwords, then lookup all users' hashes ⇒ ***O*(*M* + *N*)**
  - Salted database: Hash all passwords for each user's salt ⇒ ***O*(*MN*)**

Slow Hashes

- Solution #2: Use slower hashes
- Cryptographic hashes are usually designed to be fast
  - SHA is designed to produce a checksum of your 1 GB document as fast as possible
- Slow hash basically HMAC 10,000 times
- Password hashes are usually designed to be slow
  - Legitimate users only need to submit a few password tries. Users won't notice if it takes 0.0001 seconds or 0.1 seconds for the server to check a password.
  - Attackers need to compute millions of hashes. Using a slow hash can slow the attacker by a factor of 1,000 or more!
  - Note: We are not changing the asymptotic difficulty of attacks. We're adding a large constant factor, which can have a huge practical impact for the attacker

Slow Hashes: PBKDF2

- **Password-based key derivation function 2 (PBKDF2)**: A slow hash function
  - Setting: An underlying function that outputs random-looking bits (e.g. HMAC-SHA256)
  - Setting: The desired length of the output (*n*)
  - Setting: Iteration count (higher = hash is slower, lower = hash is faster)
  - Input: A password
  - Input: A salt
  - Output: A long, random-looking *n*-bit string derived from the password and salt
  - Implementation: Basically computing HMAC 10,000 times
- Benefits (assuming the user password is strong)
  - Derives an arbitrarily long string from the user's password
  - Output can be directly used as a symmetric key
  - Output can also be used to seed a PRNG or generate a public/private key pair
  - Algorithm is slow, but doesn't use a lot of memory (alternatives like Scrypt and Argon2 use more memory)

Offline and Online Attacks

- **Offline attack**: The attacker performs all the computation themselves
  - Example: Mallory steals the password file, and then computes hashes herself to check for matches.
  - The attacker can try a huge number of passwords (e.g., use many GPUs in parallel)
  - Defenses: Salt passwords, use slow hashes
  - If an attacker can do an offline attack, you need a really strong password (e.g. 7 or more random words)
- **Online attack**: The attacker interacts with the service
  - Example: Mallory tries to log in to a website by trying every different password. Mallory is forcing the server to compute the hashes.
  - The attacker can usually only try a few times per second, with no parallelism
  - Defenses: Add a timeout or rate limit the number of tries to prevent the attacker from trying too many times

Passwords Suck

- There is broad understanding that passwords have many problems: users choose guessable passwords, they reuse them on many sites, and users can be tricked into revealing their password (phishing emails, fake websites)
- Why are passwords still so widely used? They are easy for users to understand, familiar, and don't lock users into a particular device or technology
  - If I drop my phone in the ocean, can I still get into my accounts?
- Nonetheless, the tech community is developing better alternatives

Passkeys

Passkeys is an authentication method without passwords: it based on public-key cryptography based on standards from the FIDO Alliance such as FIDO2 and WebAuthn.

The system uses a **private key stored on your device** and a **public key stored on the server**. The private key never leaves your device.

Password managers



Why use them?

- **Enable strong passwords**, which mitigates brute-force and credential-stuffing attacks
- **Convenient login autofill**
- **Sometimes extra phishing resistance:** Many managers only autofill on the correct domain, helping detect fake sites.

Potential drawbacks:

Password managers



Why use them?

- **Enable strong passwords**, which mitigates brute-force and credential-stuffing attacks
- **Convenient login autofill**
- **Sometimes extra phishing resistance:** Many managers only autofill on the correct domain, helping detect fake sites.

Drawbacks:

- **Single point of compromise:** If the password manager account or device is compromised, many passwords may be exposed.
- **Usability recovery issues:** Losing the master password may lock you out of everything.

Email-based authentication

User receives a “magic” link in the email to login with

Why use it?

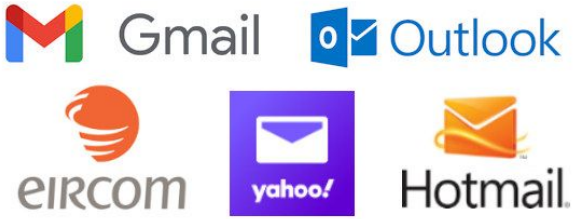
- **No additional password to remember or manage**
    a. Less credential reuse risk
- **Simple user experience:** Users just click a link in their email.

Drawbacks:

Email-based authentication

Why use it?

- **No additional password to remember or manage**
    a. Less credential reuse risk
- **Simple user experience:** Users just click a link in their email.



Drawbacks:

- **Security depends on email account security:** If the email account is compromised, all linked accounts are compromised.
    a. Attackers who gain temporary access to email can log in easily.
- **Difficult to use off-line:** Requires access to email every time.

Widely recommended to use a password manager.

Two-factor authentication

Combine two different factors of authentication, typically two out of: something I know, something I have, something I am.

A common example is password (something I know) and authenticator app push (something I have) / biometrics (something I am)

Why use it?

- Stronger than passwords alone: Attacker must steal both the password and the second factor.
- Protects against many remote attacks, for example, those trying to reset user passwords
- Widely supported

Drawbacks:

- User friction. As a result, users may accidentally approve malicious login prompts.
- Losing a device can lock users out

Widely recommended to use two-factor authentication.

Summary: Password Hashing

- Store hashes of passwords so that you can verify a user's identity without storing their password
- Attackers can use brute-force attacks to learn passwords (especially when users use weak passwords)
  - Defense: Add a different **salt** for each user: A random, public value designed to make brute-force attacks harder
- **Offline attack**: The attacker performs all the computation themselves
  - Defense: Use salted, slow hashes instead of unsalted, fast hashes
- **Online attack**: The attacker interacts with the service
  - Defense: Use timeouts
- Use a quality password manager and two-factor authentication

Cross-Site Scripting (XSS)

Textbook Chapter 22

Top 25 Most Dangerous Software Weaknesses (2020)

Rank	ID	Name	Score
[1]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	46.82
[2]	CWE-787	Out-of-bounds Write	46.17
[3]	CWE-20	Improper Input Validation	33.47
[4]	CWE-125	Out-of-bounds Read	26.50
[5]	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	23.73
[6]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	20.69
[7]	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	19.16
[8]	CWE-416	Use After Free	18.87
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	17.29
[10]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	16.44
[11]	CWE-190	Integer Overflow or Wraparound	15.81
[12]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	13.67
[13]	CWE-476	NULL Pointer Dereference	8.35
[14]	CWE-287	Improper Authentication	8.17
[15]	CWE-434	Unrestricted Upload of File with Dangerous Type	7.38
[16]	CWE-732	Incorrect Permission Assignment for Critical Resource	6.95
[17]	CWE-94	Improper Control of Generation of Code ('Code Injection')	6.53

Review: Same-Origin Policy

- Two webpages with different origins should not be able to access each other's resources
  - Example: JavaScript on **`http://evil.com`** cannot access the information on **`http://bank.com`**

Review: JavaScript

- **JavaScript**: A programming language for running code in the web browser
- JavaScript is **client-side**
  - Code sent by the server as part of the response
  - Runs in the browser, not the web server!
- Used to manipulate web pages (HTML and CSS)
  - Makes modern websites interactive
  - JavaScript can be directly embedded in HTML with `<script>` tags
- Most modern webpages involve JavaScript
  - JavaScript is supported by all modern web browsers
- You don’t need to know JavaScript syntax
  - However, knowing common attack functions helps

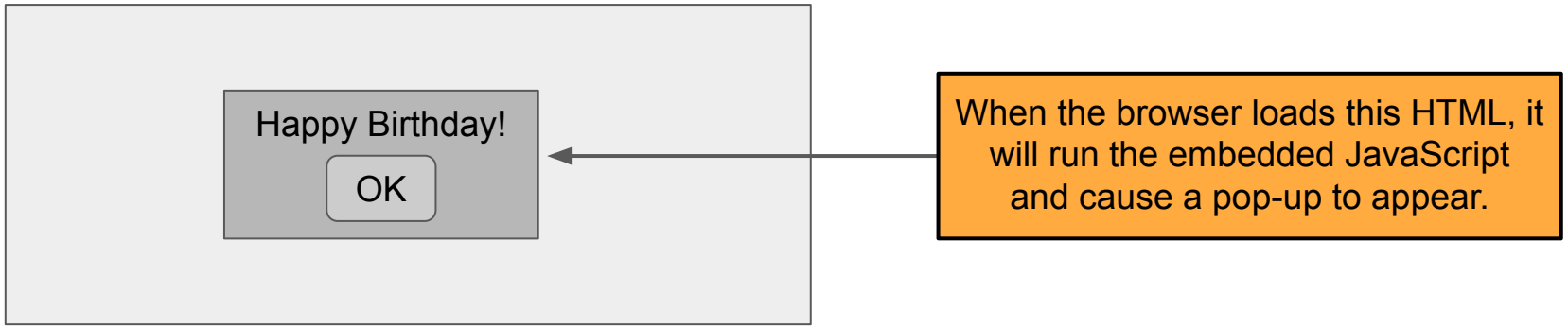
Review: JavaScript

- JavaScript can create a pop-up message

HTML (with embedded JavaScript)

```
<script>alert("Happy Birthday!")</script>
```

Webpage



A Go HTTP Handler

Handler

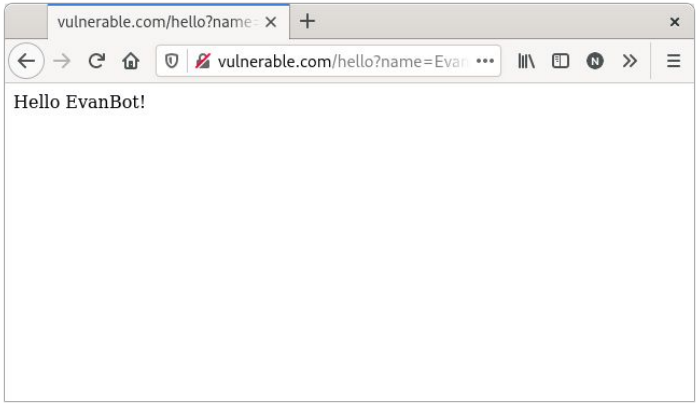
```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=EvanBot
```

Response

```
<html><body>Hello EvanBot!</body></html>
```



A Go HTTP Handler

Handler

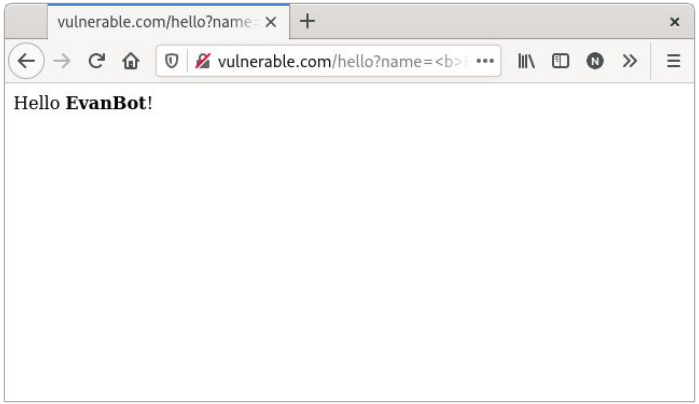
```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=<b>EvanBot</b>
```

Response

```
<html><body>Hello <b>EvanBot</b>!</body></html>
```



A Go HTTP Handler

Problem: This input represents control data (HTML), not just text!

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello <script>alert(1)</script>!</body></html>
```



A Go HTTP Handler

Not just `%s`: It can happen with any string manipulation

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    content := "<html><body>Hello "+name+"!</body></html>"
    fmt.Fprint(w, content)
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello <script>alert(1)</script>!</body></html>
```



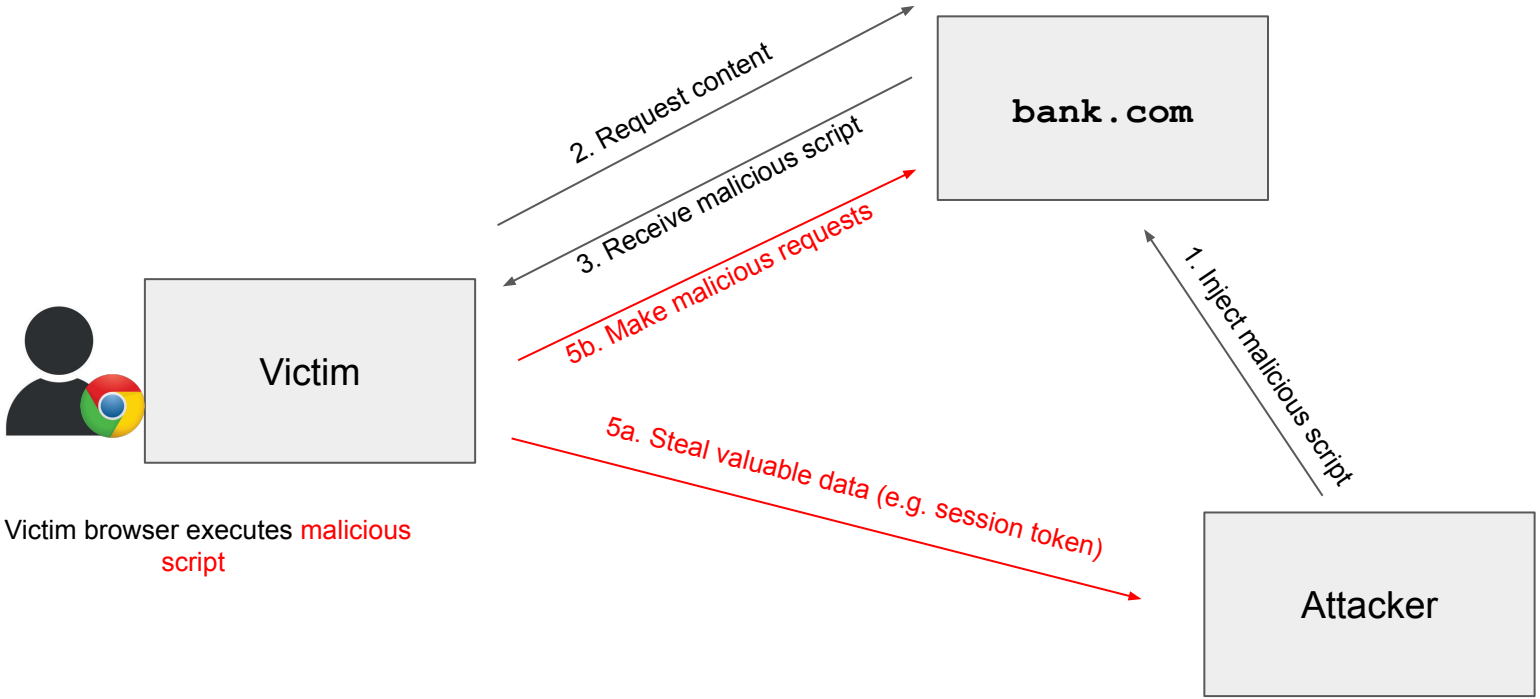
Cross-Site Scripting (XSS)

- Idea: The attacker adds malicious JavaScript to a legitimate website
  - The legitimate website will send the attacker's JavaScript to browsers
  - The attacker's JavaScript will run with the origin of the legitimate website
  - Now the attacker's JavaScript can access information on the legitimate website!
- **Cross-site scripting** (**XSS**): Injecting JavaScript into websites that are viewed by other users
  - Cross-site scripting subverts the same-origin policy
- Two main types of XSS
  - Stored XSS
  - Reflected XSS

Stored XSS

- **Stored XSS** (**persistent XSS**): The attacker's JavaScript is **stored** on the legitimate server and sent to browsers
- Classic example: Facebook pages
  - An attacker puts some JavaScript on their Facebook page
  - Anybody who loads the attacker's page will see JavaScript (with the origin of Facebook)
- Remember: Stored XSS is a server-side vulnerability!

Stored XSS

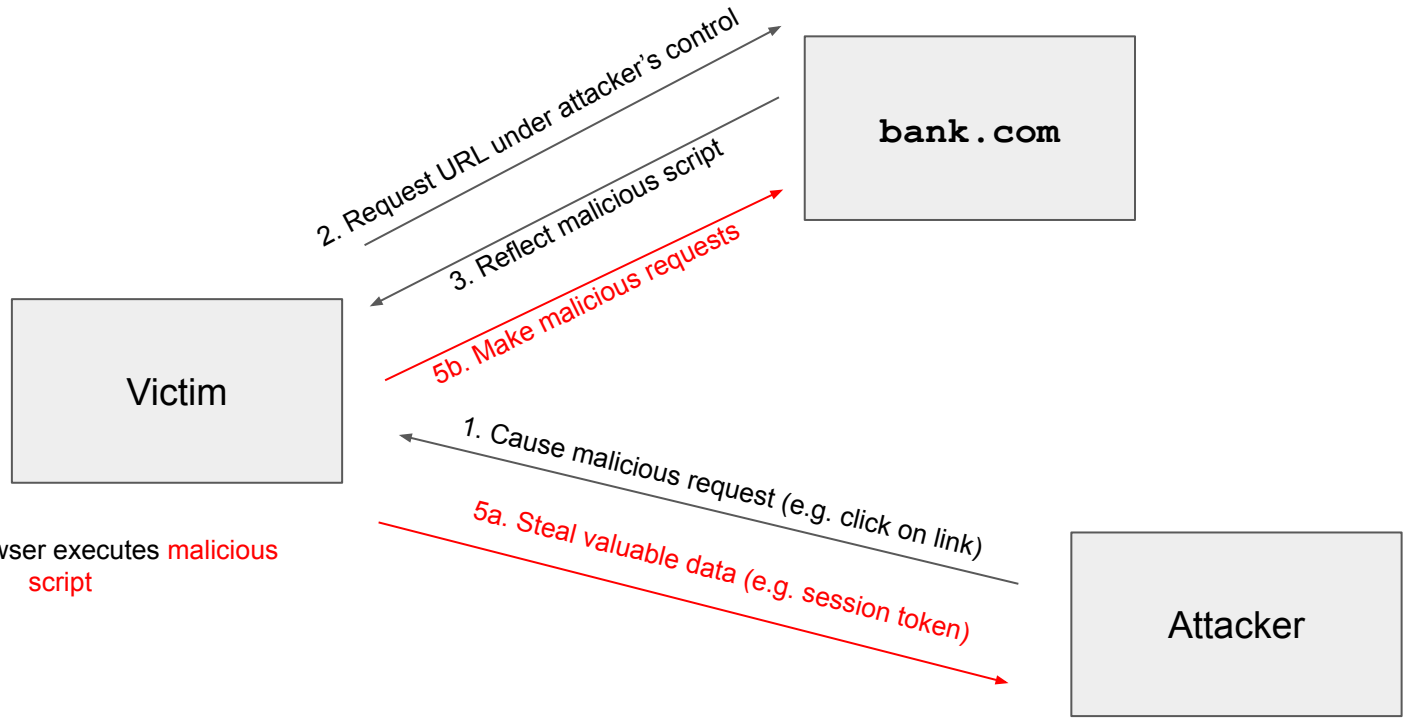


4. Victim browser executes malicious script

Reflected XSS

- **Reflected XSS**: The attacker causes the victim to input JavaScript into a request, and the content is **reflected** (copied) in the response from the server
- Classic example: Search
  - If you make a request to `http://google.com/search?q=evanbot`, the response will say "10,000 results for evanbot"
  - If you make a request to `http://google.com/search?q=<script>alert(1)</script>`, the response will say "10,000 results for `<script>alert(1)</script>`"
- Reflected XSS requires the victim to make a request with injected JavaScript

Reflected XSS



4. Victim browser executes malicious script

Reflected XSS: Making a Request

- How do we force the victim to make a request to the legitimate website with injected JavaScript?
  - Trick the victim into visiting the attacker's website, and include an embedded iframe that makes the request
    - Can make the iframe very small (1 pixel x 1 pixel), so the victim doesn't notice it:

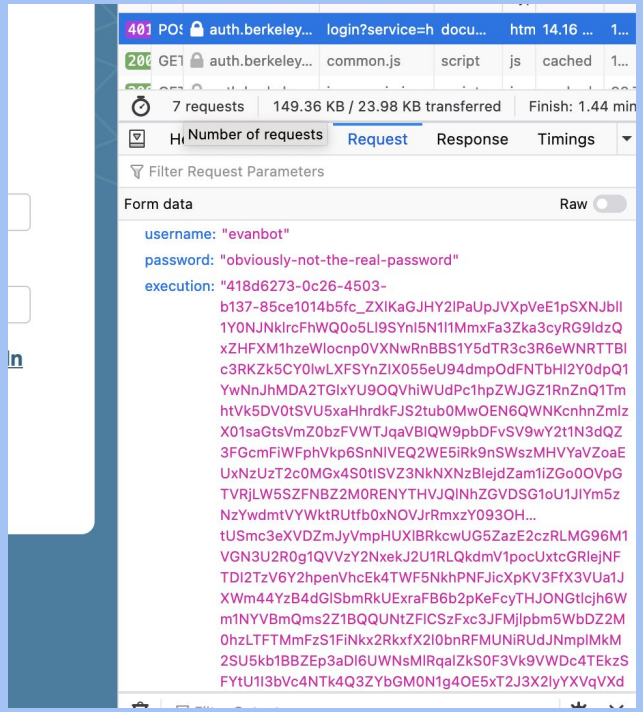
      ```
      <iframe height=1 width=1
      src="http://google.com/search?q=<script>alert(1)</script>">
      ```
  - Trick the victim into clicking a link (e.g. posting on social media, sending a text, etc.)
  - Trick the victim into visiting the attacker's website, which redirects to the reflected XSS link
  - … and many more!

Reflected XSS is not CSRF

- Reflected XSS and CSRF both require the victim to make a request to a link
  - XSS: An HTTP response contains maliciously inserted JavaScript, executed on the client side
  - CSRF: A malicious HTTP request is made (containing the user's cookies), executing an effect on the server side

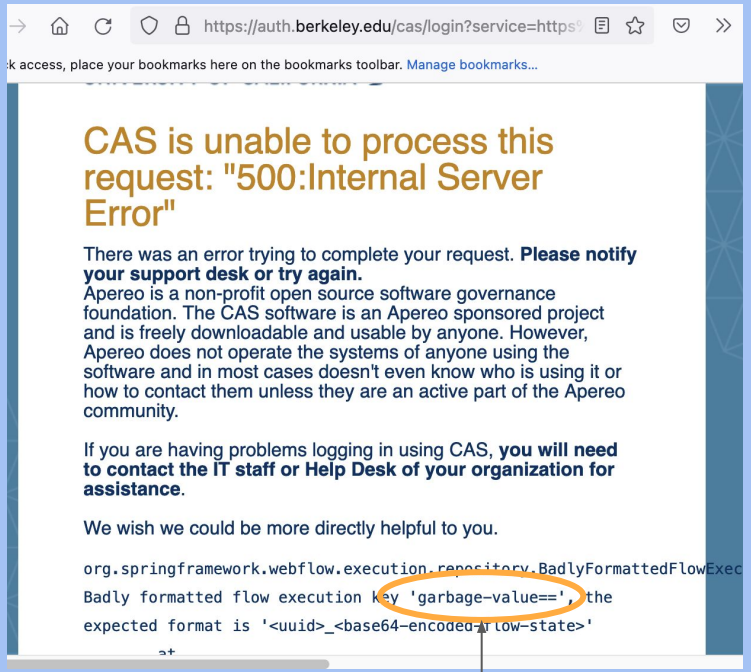
XSS in the Wild… On CalNet?!

- In 2021, 61C student Rohan Mathur was exploring the CalNet login page
- Fields submitted when logging in:
  - **`username`**: What user am I logging in as?
  - **`password`**: What's the user's password?
  - **`execution`**: Server-side state (like a CSRF token with extra data)



XSS in the Wild… On CalNet?!

- The server expects **`execution`** to be in a specific format that contains its server-side state
  - What if you muck with **`execution`**?
- The "corrupted" execution key is placed into the error message in the HTML to aid debugging
  - … and CalNet at the time did not escape the execution key



Constructing an Attack on CalNet

Attack: Force a POST request to CalNet!

```
<html>
  <head>
    <script>
      // When the malicious page finishes loading, automatically submit the form!
      document.addEventListener('DOMContentLoaded', () => {
        document.getElementById('form').submit();
      });
    </script>
  </head>
  <body>
    <!-- Malicious form containing our malicious execution data. -->
    <form id="form" action="https://auth.berkeley.edu/cas/login" method="POST">
      <input name="username" type="text" value="evanbot" />
      <input name="password" type="text" value="obviously-not-the-real-password" />
      <input name="execution" type="text" value="<script>alert('XSS!')</script>" />
    </form>
  </body>
</html>
```

So What Happened?

- CalNet also accepts **`execution`** parameters over URL query parameters
  - A link like **`https://auth.berkeley.edu/cas/login?execution=<script>alert('XSS!')</script>`** would result in the same attack
- It would only fire if you clicked the button to show the error
- … But if clicked, the injected JavaScript could
  - Present a fake login prompt
  - Steal your CalNet password
  - Log you in as the attacker's account
  - Steal your authentication session token
- Root cause: A >5 year old sample page modified by CalNet
  - CalNet uses software from Apereo, which comes with sample pages for logging in
  - Apereo fixed that bug many years before, but sample pages don't get included in bug fixes!

XSS Defenses

- Defense: **HTML sanitization**
  - Idea: Certain characters are special, so server should remove those characters or encode them in a way that will render them harmless
- Escaping: e.g., replace < with `<` with " with `"`
- Rely on trusted libraries to do this for you

```
<html>
<body>
Hello &lt;script&gt;alert(1)&lt;/script&gt;!
</body>
</html>
```

XSS Defenses: Escaping

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", html.EscapeString(name))
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello &lt;script&gt;alert(1)&lt;/script&gt;!</body></html>
```

XSS Defenses: Escaping

- If a programmer has to take an extra action for every usage…
  - They are eventually going to forget (e.g., CalNet)
  - Consider human factors!
- Nowadays, escaping is generally achieved through **templating**
  - HTML templates are essentially their own language, where you declare what data goes where
  - The templating engine handles all the escaping internally, automatically

```
<html>
<body>
Hello {{.name}}!
</body>
</html>
```

Example: Golang HTML template

XSS Defenses: CSP

- Defense: **Content Security Policy** (**CSP**)
  - Idea: Instruct the browser to only use resources loaded from specific places
  - Uses additional headers to specify the policy
- Standard approach:
  - Disallow all inline scripts (JavaScript code directly in HTML), which prevents inline XSS
    - Example: Disallow `<script>alert(1)</script>`
  - Only allow scripts from specified domains, which prevents XSS from linking to external scripts
    - Example: Disallow `<script src="https://cs161.org/hack.js">`
- Also works with other content (e.g. iframes, images, etc.)
- Relies on the browser to enforce security, so more of a mitigation for defense-in-depth